46. Design E-commerce Platform

Key Terms:

Term	Full Name	What it does
SKU	Stock Keeping Unit	Unique product variant identifier
OMS	Order Management System	Handles order lifecycle
WMS	Warehouse Management System	Manages inventory and fulfillment
PDP	Product Detail Page	Individual product page
PLP	Product Listing Page	Category/search results page
Cart Abandonment	Incomplete checkout	User adds items but doesn't purchase
Flash Sale	Time-limited sale	High traffic spike (10-100x normal)
Faceted Search	Filter by attributes	Search with category/price/brand filters

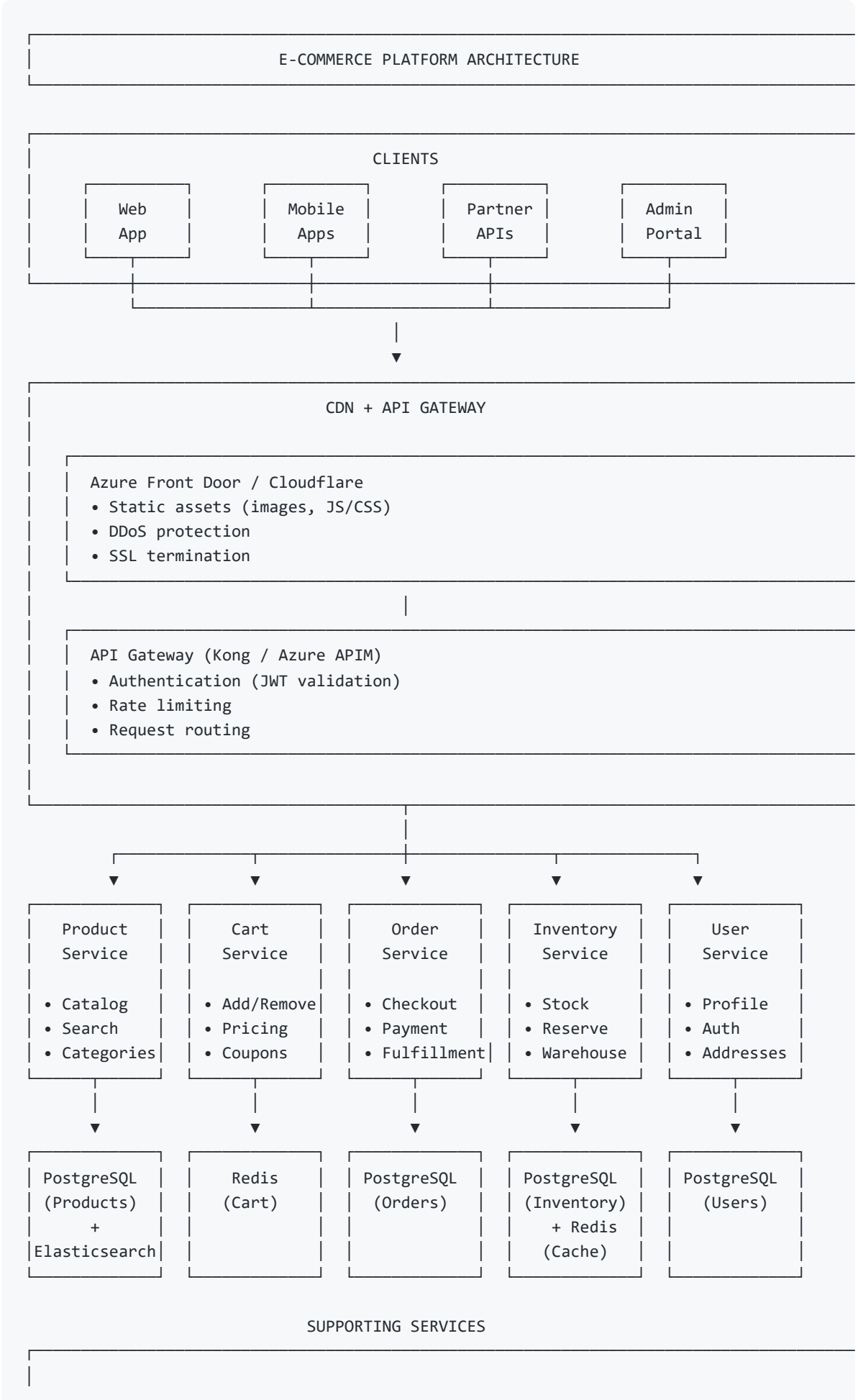
Problem Statement

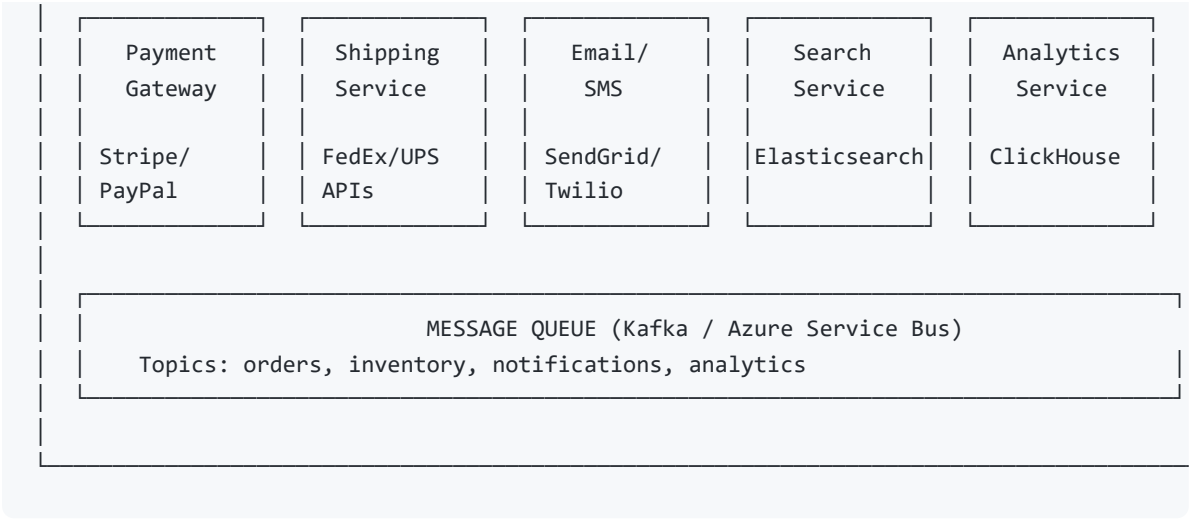
Design the backend for a large-scale e-commerce platform like Amazon, handling product catalog, shopping cart, checkout, orders, and inventory management.

Clarifying Questions to Ask

1. **Scale**: How many products? Daily orders? Peak traffic? (1M products, 100K orders/day?)
2. **Geography**: Single region or global? Multi-currency?
3. **Inventory**: Single warehouse or distributed? Real-time stock?
4. **Checkout**: Payment processing in-house or third-party?
5. **Search**: Full-text search with filters/facets?
6. **Personalization**: Product recommendations?
7. **Flash sales**: Handle traffic spikes (10x-100x normal)?

High-Level Design


E-COMMERCE PLATFORM ARCHITECTURE
CLIENTS
Web App
Mobile Apps
Partner APIs
Admin Portal
CDN + API GATEWAY
Azure Front Door / Cloudflare
• Static assets (images, JS/CSS)
• DDoS protection
• SSL termination
API Gateway (Kong / Azure APIM)
• Authentication (JWT validation)
• Rate limiting
• Request routing
Product Service
• Catalog
• Search
• Categories
Cart Service
• Add/Remove
• Pricing
• Coupons
Order Service
• Checkout
• Payment
• Fulfillment
Inventory Service
• Stock
• Reserve
• Warehouse
User Service
• Profile
• Auth
• Addresses
PostgreSQL (Products) + Elasticsearch
Redis (Cart)
PostgreSQL (Orders)
PostgreSQL (Inventory) + Redis (Cache)
PostgreSQL (Users)
SUPPORTING SERVICES



API Design

Product APIs

Method	Endpoint	Description
`GET`	`/api/v1/products?category={cat}&page={n}`	List products
`GET`	`/api/v1/products/{productId}`	Get product details
`GET`	`/api/v1/products/search?q={query}`	Search products

Cart APIs

Method	Endpoint	Description
`GET`	`/api/v1/cart`	Get current cart
`POST`	`/api/v1/cart/items`	Add item to cart
`PUT`	`/api/v1/cart/items/{itemId}`	Update item quantity
`DELETE`	`/api/v1/cart/items/{itemId}`	Remove item from cart

Order APIs

Method	Endpoint	Description
`POST`	`/api/v1/orders`	Place an order (from cart)
`GET`	`/api/v1/orders/{orderId}`	Get order details
`GET`	`/api/v1/orders?status={status}`	List user orders

`POST`	`/api/v1/orders/{orderId}/cancel`	Cancel order
`GET`	`/api/v1/orders/{orderId}/tracking`	Get shipping tracking

Payment APIs

Method	Endpoint	Description
`POST`	`/api/v1/payments`	Process payment
`POST`	`/api/v1/payments/{paymentId}/refund`	Initiate refund

Inventory APIs (Internal)

Method	Endpoint	Description
`GET`	`/api/v1/inventory/{productId}`	Check stock availability
`PUT`	`/api/v1/inventory/{productId}/reserve`	Reserve inventory
`PUT`	`/api/v1/inventory/{productId}/release`	Release reservation

Product Service

```
-- Product catalog schema
CREATE TABLE products (
    id BIGINT PRIMARY KEY,
    sku VARCHAR(50) UNIQUE,
    name VARCHAR(255),
    description TEXT,
    category_id BIGINT,
    brand_id BIGINT,
    price DECIMAL(10, 2),
    compare_at_price DECIMAL(10, 2),  -- Original price for discounts
    currency VARCHAR(3) DEFAULT 'USD',
    status VARCHAR(20),  -- active, draft, archived
    attributes JSONB,    -- {"color": "red", "size": "M"}
    images JSONB,        -- ["url1", "url2"]
    created_at TIMESTAMP,
    updated_at TIMESTAMP
);

CREATE TABLE categories (
    id BIGINT PRIMARY KEY,
    parent_id BIGINT REFERENCES categories(id),
    name VARCHAR(100),
    slug VARCHAR(100),
    path LTREE  -- PostgreSQL extension for hierarchical data
```

```
);

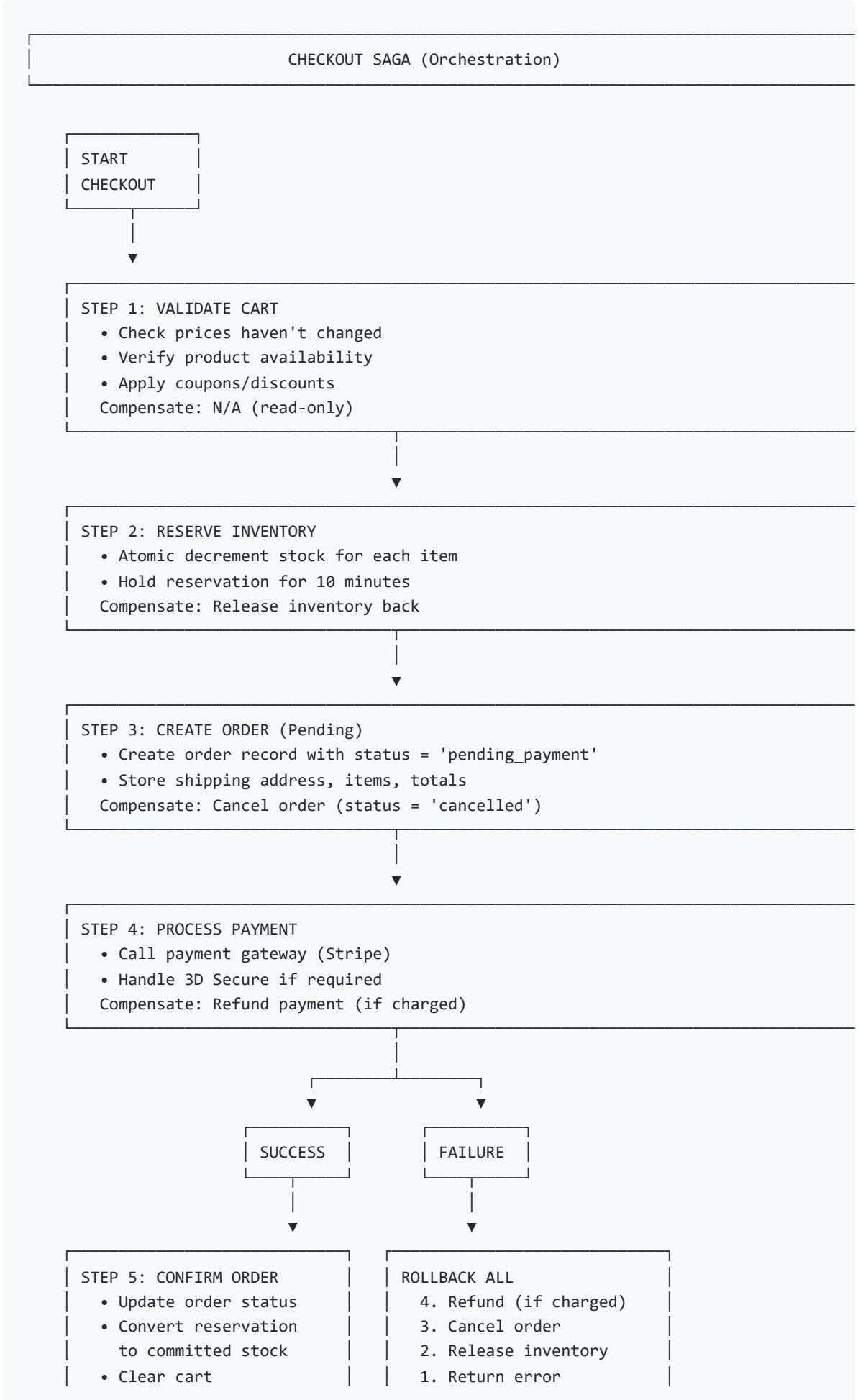
-- Elasticsearch index for search
{
  "mappings": {
    "properties": {
      "name": {"type": "text", "analyzer": "standard"},
      "description": {"type": "text"},
      "category": {"type": "keyword"},
      "brand": {"type": "keyword"},
      "price": {"type": "float"},
      "attributes": {"type": "nested"},
      "in_stock": {"type": "boolean"}
    }
  }
}
```

Shopping Cart (Redis)

```
CART DATA MODEL (Redis Hash):
┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│                                                                                              │
│   KEY: cart:{user_id}                                                                        │
│                                                                                              │
│   HASH FIELDS:                                                                               │
│     item:{product_id} → {"qty": 2, "price": 29.99, "added_at": "..."}                        │
│     item:{product_id} → {"qty": 1, "price": 49.99, "added_at": "..."}                        │
│     coupon → "SAVE10"                                                                        │
│     updated_at → "2024-01-15T10:30:00Z"                                                      │
│                                                                                              │
│   EXPIRATION: 7 days (TTL)                                                                   │
│                                                                                              │
│   Commands:                                                                                  │
│     HSET cart:123 item:456 '{"qty": 2, "price": 29.99}'                                      │
│     HGETALL cart:123                                                                         │
│     HDEL cart:123 item:456                                                                   │
│     EXPIRE cart:123 604800  # 7 days                                                         │
│                                                                                              │
└──────────────────────────────────────────────────────────────────────────────────────────────┘

ANONYMOUS → LOGGED-IN MERGE:
┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│  When user logs in:                                                                          │
│    1. Fetch anonymous cart: cart:anon_{session_id}                                           │
│    2. Fetch user cart: cart:user_{user_id}                                                   │
│    3. Merge (user cart wins on conflicts)                                                    │
│    4. Delete anonymous cart                                                                  │
└──────────────────────────────────────────────────────────────────────────────────────────────┘
```

Checkout Flow (Saga Pattern)


CHECKOUT SAGA (Orchestration)
START CHECKOUT
STEP 1: VALIDATE CART
• Check prices haven't changed
• Verify product availability
• Apply coupons/discounts
Compensate: N/A (read-only)
STEP 2: RESERVE INVENTORY
• Atomic decrement stock for each item
• Hold reservation for 10 minutes
Compensate: Release inventory back
STEP 3: CREATE ORDER (Pending)
• Create order record with status = 'pending_payment'
• Store shipping address, items, totals
Compensate: Cancel order (status = 'cancelled')
STEP 4: PROCESS PAYMENT
• Call payment gateway (Stripe)
• Handle 3D Secure if required
Compensate: Refund payment (if charged)
SUCCESS
FAILURE
STEP 5: CONFIRM ORDER
• Update order status
• Convert reservation to committed stock
• Clear cart
ROLLBACK ALL
4. Refund (if charged)
3. Cancel order
2. Release inventory
1. Return error

```
    │   • Send confirmation email │   │                             │
    └─────────────────────────────┘   └─────────────────────────────┘
```

Inventory Management

```
┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│                                INVENTORY TRACKING                                            │
└──────────────────────────────────────────────────────────────────────────────────────────────┘

DATABASE SCHEMA:
┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│                                                                                              │
│  CREATE TABLE inventory (                                                                    │
│      product_id BIGINT,                                                                      │
│      warehouse_id BIGINT,                                                                    │
│      available_qty INT,      -- Can sell                                                     │
│      reserved_qty INT,       -- Reserved for pending orders                                  │
│      on_hand_qty INT,        -- Physical stock = available + reserved                        │
│      PRIMARY KEY (product_id, warehouse_id)                                                  │
│  );                                                                                          │
│                                                                                              │
│  CONSTRAINT: available_qty >= 0                                                              │
│  CONSTRAINT: on_hand_qty = available_qty + reserved_qty                                      │
│                                                                                              │
└──────────────────────────────────────────────────────────────────────────────────────────────┘

RESERVATION PATTERN:
┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│                                                                                              │
│  -- Reserve inventory (atomic)                                                               │
│  UPDATE inventory                                                                            │
│  SET available_qty = available_qty - @qty,                                                   │
│      reserved_qty = reserved_qty + @qty                                                      │
│  WHERE product_id = @pid                                                                     │
│    AND warehouse_id = @wid                                                                   │
│    AND available_qty >= @qty;  -- Prevent negative                                           │
│                                                                                              │
│  -- On order completion: Commit reservation                                                  │
│  UPDATE inventory                                                                            │
│  SET reserved_qty = reserved_qty - @qty,                                                     │
│      on_hand_qty = on_hand_qty - @qty                                                        │
│  WHERE product_id = @pid AND warehouse_id = @wid;                                            │
│                                                                                              │
│  -- On order cancellation: Release reservation                                               │
│  UPDATE inventory                                                                            │
│  SET available_qty = available_qty + @qty,                                                   │
│      reserved_qty = reserved_qty - @qty                                                      │
│  WHERE product_id = @pid AND warehouse_id = @wid;                                            │
│                                                                                              │
└──────────────────────────────────────────────────────────────────────────────────────────────┘

REDIS CACHE (Hot products):
┌──────────────────────────────────────────────────────────────────────────────────────────────┐
```

```
│
│  # Fast availability check
│  GET inventory:{product_id}:{warehouse_id}
│  → "45"  (available quantity)
│
│  # Atomic decrement for high-traffic products
│  DECRBY inventory:{product_id}:{warehouse_id} 1
│  → If result < 0, rollback and return "out of stock"
│
│  # Sync DB → Redis every few seconds
│
└──────────────────────────────────────────────────────────────────────────────────────
```

Flash Sale Handling

```
┌──────────────────────────────────────────────────────────────────────────────────────
│                              FLASH SALE ARCHITECTURE
└──────────────────────────────────────────────────────────────────────────────────────

PROBLEM: 100x traffic spike for limited inventory (1000 items, 100K users)

SOLUTION:
┌──────────────────────────────────────────────────────────────────────────────────────
│
│  1. QUEUE-BASED THROTTLING
│     ┌──────────────┐     ┌──────────────┐     ┌──────────────┐
│     │   Users      │────▶│   Queue      │────▶│  Workers     │
│     │  (100K)      │     │  (Redis)     │     │  (Process    │
│     │              │     │              │     │   1K/sec)    │
│     └──────────────┘     └──────────────┘     └──────────────┘
│
│  2. PRE-GENERATE TOKENS
│     • Generate 1000 purchase tokens before sale
│     • First 1000 users get tokens, others get "sold out"
│     • Token holders have 10 minutes to complete purchase
│
│  3. REDIS STOCK COUNTER
│     SETNX flash:sale:123:stock 1000  # Set only if not exists
│     DECR flash:sale:123:stock        # Atomic decrement
│     → If result < 0, return "sold out"
│
│  4. RATE LIMITING
│     • 1 request per user per second
│     • CAPTCHA for suspicious traffic
│
└──────────────────────────────────────────────────────────────────────────────────────
```

Key Components

Service	Purpose	Database	Key Features

Product	Catalog management	PostgreSQL + Elasticsearch	Full-text search, facets
Cart	Shopping cart	Redis	TTL, merge on login
Order	Order lifecycle	PostgreSQL	Saga, idempotency
Inventory	Stock management	PostgreSQL + Redis	Reservation pattern
Payment	Payment processing	- (External: Stripe)	Idempotent, retry
Search	Product search	Elasticsearch	Facets, autocomplete
User	Auth, profiles	PostgreSQL	OAuth, JWT

Scalability Considerations

- **Read replicas**: Product catalog is read-heavy; use replicas
- **Caching**: CDN for images, Redis for product details
- **Database sharding**: Shard orders by user_id
- **Async processing**: Order confirmation emails via queue
- **Search scaling**: Elasticsearch cluster with shards/replicas
- **Flash sales**: Queue-based throttling, pre-generated tokens

Trade-offs

Decision	Option A	Option B	Recommendation
Cart storage	Database	Redis	**Redis** - ephemeral, fast
Checkout	2PC	Saga	**Saga** - better failure handling
Search	PostgreSQL FTS	Elasticsearch	**ES** - facets, relevance
Inventory	Eventual	Real-time	**Real-time** - avoid overselling
Payment	Synchronous	Async	**Sync** - user expects immediate feedback

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Redis	Shopping cart storage, inventory caching, flash sale counters	Tech Guide
PostgreSQL	Product catalog, orders, inventory, and user data persistence	Tech Guide
Kafka	Async messaging for orders, inventory updates, and notifications	Tech Guide
Elasticsearch	Full-text product search with faceted filtering	Tech Guide
Saga Pattern	Checkout orchestration: reserve → order → charge → confirm with compensations	Tech Guide
CDN	Static assets, product images, and edge caching	Tech Guide

How to Explain It Well

> **Opening**: "E-commerce needs to handle catalog browsing, cart management, and checkout atomically. I'll use microservices with a Saga pattern for distributed transactions."

> **Key talking points**:
>
> - "Redis for cart - ephemeral data, fast operations, automatic expiry"
> - "Saga pattern for checkout: reserve → create order → charge → confirm, with compensations"
> - "Inventory uses reservation pattern: separate available vs reserved quantities"
> - "Elasticsearch for search with faceted filtering (category, price, brand)"
> - "Flash sales: Redis atomic counters + queue-based throttling"
> - "Idempotency keys for payment retries - charge exactly once"

Common Follow-up Questions

1. **How do you handle overselling?**
    - Atomic inventory reservation (UPDATE with WHERE qty >= needed)
    - Redis counters for flash sales
    - Compensation: refund + notify customer
2. **How do you handle abandoned carts?**

- Redis TTL (7 days)
- Send reminder email after 1 hour, 24 hours
- Release inventory reservation after 10 minutes

3. **How do you handle price changes during checkout?**

- Lock price at cart addition
- Re-validate at checkout
- Show user if price changed, require confirmation